ADITYA UNIVERSITY

Project Review 1&2

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PROJECT TITLE : COLLEGE EVENT MANAGEMENT SYSTEM

Guide:- T.Vijayalakshmi
Assistant Professor
Department Of AIML

Team Members :	Registration Number:	Branch:
B.LASYA SHARON	25B11AI098	AIML
ANANYA DAS	25B11AI037	AIML
SANANDA CHOWDHURY	25B11AIA67	AIML
MEDHA		
K.KURMANADH	25B11AI614	AIML

ADITYA UNIVERSITY

Outline:

- Project Title & Team Members
- Project Objective
- Problem Statement
- System Modules
- Class Structure & OOP Concepts
- Event Registration & Attendance
- Venue Allocation
- Core Functions & STL Vector
- File Handling & Data Persistence
- Input Validation
- Testing & Validation
- Application Output
- Team Member Contributions
- Future Enhancements
- Conclusion

ADITYA UNIVERSITY

ABSTRACT:

- The College Event Management System is a C++ application designed to manage common college-event activities in a structured way. It maintains student, coordinator, venue and event information, supports event registration, venue allocation and attendance marking, and generates an event-wise report.

- The project applies Object-Oriented Programming concepts such as classes, inheritance, abstraction, encapsulation and polymorphism. STL vectors are used to store dynamic collections, while file handling is used to save and reload data between program runs.

ADITYA UNIVERSITY

INTRODUCTION:

- College events involve multiple activities such as creating events, assigning coordinators and venues, registering students and tracking attendance. Managing these activities manually can make information difficult to organize and retrieve.

- This project provides a simple console-based system that brings these operations together in one application and demonstrates how C++ OOP can model a real-world event-management workflow.

ADITYA UNIVERSITY

PROBLEM STATEMENT:

- Create a single system to manage college students, coordinators, venues and events.
- Allow students to register for available events without duplicate registration.
- Assign available venues to events and prevent reuse of an already allocated venue.
- Record attendance only for registered students.
- Generate event-wise reports showing registration and attendance status.
- Store data in files so records can be loaded again when the program starts.

ADITYA UNIVERSITY

OBJECTIVES:

- To develop a College Event Management System using C++.
- To apply OOP concepts such as inheritance, abstraction, encapsulation and polymorphism.
- To manage students, coordinators, venues and events using classes.
- To provide event registration and attendance tracking.
- To allocate venues while checking availability.
- To generate useful event reports.
- To save and load system data using file handling.

ADITYA UNIVERSITY

SYSTEM MODULES

Module	Purpose
Student Management	Add and display students; store registered event IDs.
Coordinator Management	Add and display event coordinators.
Venue Management	Create venues, show capacity and allocation status.
Event Management	Create and display events with date, coordinator and capacity.
Venue Allocation	Assign an available venue to a selected event.
Event Registration	Register a student and prevent duplicate/full registrations.
Attendance	Mark registered students as present.
Event Report	Display registered students with Present/Absent status.
File Storage	Save and reload students, coordinators, venues and events.

ADITYA UNIVERSITY

CLASS STRUCTURE

Person – abstract base class containing common ID and name data.

Student : public Person – stores department and registered event IDs.

Coordinator : public Person – stores department information.

Venue – stores venue ID, name, capacity and allocation status.

Event – stores event details, venue, registered students and attendance.

EventManagementSystem – manages collections and performs the main operations.

ADITYA UNIVERSITY

C++ CONCEPTS USED

Concept	Where Used	Purpose
Classes & Objects	Person, Student, Coordinator, Venue, Event	Model real-world entities
Inheritance	Student/Coordinator inherit Person	Reuse common attributes
Abstraction	Person has pure virtual display()	Define common interface
Polymorphism	virtual display()	Different display behavior
Encapsulation	Private class data + methods	Protect and organize data
Constructors	All major classes	Initialize objects
STL vector	Students, events, venues, IDs	Dynamic collections
Algorithms	find()	Search and duplicate checking

ADITYA UNIVERSITY

ATTENDANCE & EVENT REPORT

```
• bool present = find(pres.begin(),
  pres.end(), reg[i])
                  != pres.end();

  cout << students[j].getId() << " "
       << students[j].getName() << " - "
       << (present ? "Present" :
  "Absent");
```

- Attendance can be marked only for a student who is registered for the event.
- The system prevents the same student from being marked present twice.
- Event reports display the event name, date, coordinator and registration count.
- Each registered student is shown with Present or Absent status.

ADITYA UNIVERSITY

VENUE ALLOCATION

```
if (v->isAllocated()) {
    cout << "Venue already
allocated.\n";
    return;
}
v->allocate();
e->setVenueId(vid);
```

- Display available events and venues.
- Accept an Event ID and Venue ID.
- Check whether both records exist.
- Check whether the selected venue is already allocated.
- Allocate the venue and store its ID inside the event.
- If any condition fails, the system displays an appropriate message.

ADITYA UNIVERSITY

CORE FUNCTIONS

Function	Role
addStudent()	Creates a student record and assigns an ID.
addCoordinator()	Creates a coordinator record.
addVenue()	Creates a venue with capacity.
addEvent()	Creates an event with date, coordinator and participant limit.
allocateVenueToEvent()	Assigns an available venue to an event.
registerStudentForEvent()	Validates and registers a student.
markAttendance()	Marks a registered student present.
generateEventReport()	Displays event registration and attendance.
saveData() / loadData()	Persist and restore application data.

ADITYA UNIVERSITY

STL VECTOR & REGISTRATION

```
vector<Student> students;
vector<Event> events;

bool addRegistration(int sid) {
    if ((int)registeredStudents.size()
>= maxParticipants)
        return false;
    if
(find(registeredStudents.begin(),
              registeredStudents.end(),
sid)
        != registeredStudents.end())
        return false;
    registeredStudents.push_back(sid);
    return true;

}
```

- The vector stores a dynamic list of objects or student IDs. The find() algorithm is used to check whether a student is already registered before adding a new registration.

ADITYA UNIVERSITY

FILE HANDLING & DATA PERSISTENCE

```
ofstream fs("students.txt");
for (size_t i = 0; i < students.size();
i++)
    fs << students[i].serialize() <<
"\n";

ofstream fe("events.txt");
for (size_t i = 0; i < events.size();
i++)
    fe << events[i].serialize() <<
"\n";
```

The system writes records to separate text files: students.txt, coordinators.txt, venues.txt and events.txt. loadData() reads these files when the program starts and reconstructs the stored objects.

ADITYA UNIVERSITY

DATA SERIALIZATION

```
string serialize() const {
    stringstream ss;
    ss << eventId << "|" << eventName
<< "|"
       << eventDate << "|" <<
coordinatorName << "|"
       << venueId << "|" <<
maxParticipants << "|";
    ...
    return ss.str();
}
```

Serialization converts an object's important fields into a text format that can be written to a file. splitStr() later separates the stored values and rebuilds the objects.

ADITYA UNIVERSITY

INPUTS

Operation	Main Input	Example
Add Student	Name, Department	Sananda, AIML
Add Coordinator	Name, Department	Event Coordinator, AIML
Add Venue	Venue Name, Capacity	Seminar Hall, 200
Create Event	Name, Date, Coordinator, Max Participants	Tech Fest, 15-10-2026
Allocate Venue	Event ID, Venue ID	1, 1
Register Student	Student ID, Event ID	1, 1
Attendance	Event ID, Student ID	1, 1
Event Report	Event ID	1

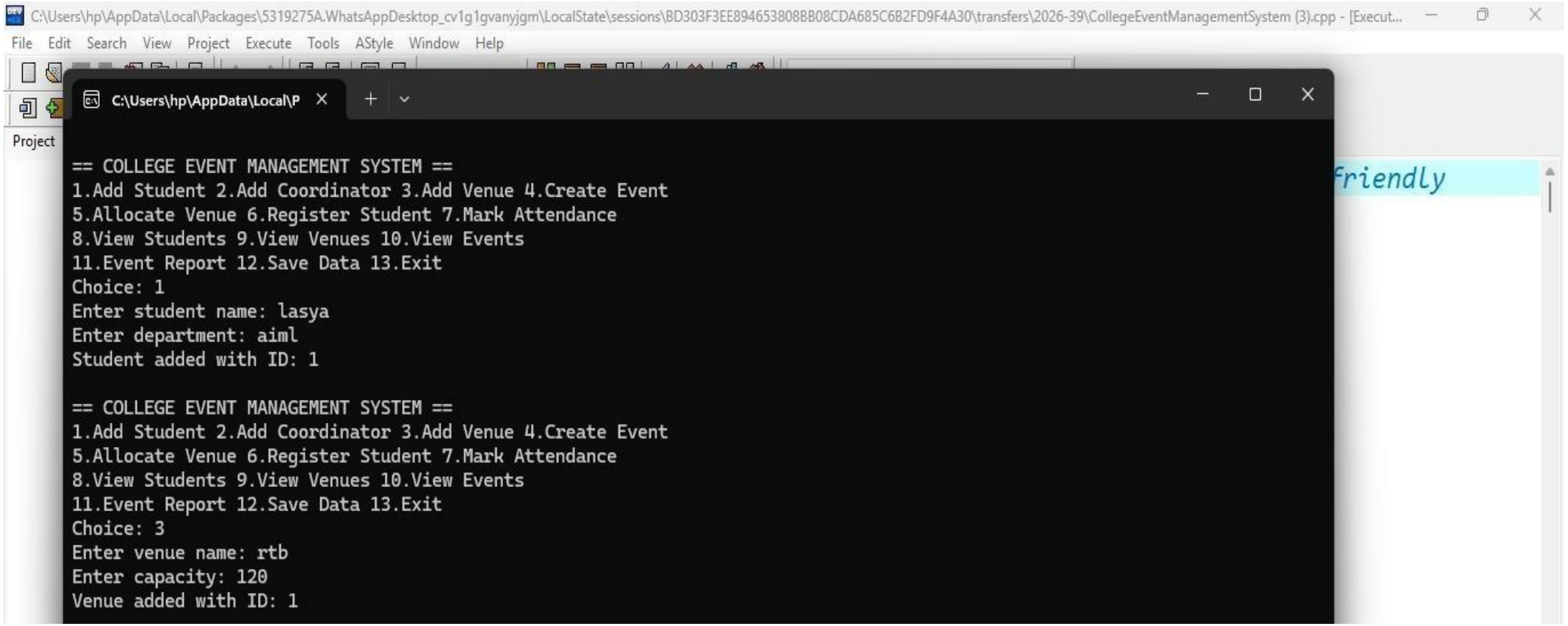
ADITYA UNIVERSITY

INPUT VALIDATION & ERROR HANDLING

- Invalid menu input is detected using cin.fail().
- Event and student IDs are checked before operations.
- Duplicate student registration is rejected.
- Registration is rejected when an event reaches maximum capacity.
- Venue allocation is rejected if the venue is already allocated.
- Attendance is rejected when the student is not registered or is already marked present.
- Malformed file lines are skipped while loading data.

ADITYA UNIVERSITY

APPLICATION OUTPUT



ADITYA UNIVERSITY

ALGORITHM:

- **Start** the program and create EventManagementSystem object.
- Load previously saved data from text files.
- Display the main menu.
- Read the user's choice.
- Perform the selected operation: add, allocate, register, attendance, view, report or save.
- Display the result or validation message.
- Repeat the menu until the user selects **Exit.**
- Save data and **stop** the program.

ADITYA UNIVERSITY

TESTING AND VALIDATION

Test	Input / Situation	Expected Result
1	Create student	Student added with generated ID
2	Create event	Event created with generated ID
3	Allocate free venue	Venue allocated to event
4	Allocate same venue again	Venue already allocated
5	Register valid student	Registered successfully
6	Register same student twice	Already registered
7	Register after capacity	Registration failed: event full
8	Mark registered student	Attendance marked
9	Mark unregistered student	Attendance rejected
10	Restart program	Previously saved records are loaded
11	Invalid menu input	Invalid input message

ADITYA UNIVERSITY

APPLICATION MENU

```
== COLLEGE EVENT MANAGEMENT SYSTEM ==
1. Add Student
2. Add Coordinator
3. Add Venue
4. Create Event
5. Allocate Venue
6. Register Student
7. Mark Attendance
8. View Students
9. View Venues
10. View Events
11. Event Report
12. Save Data
13. Exit
```

ADITYA UNIVERSITY

TEAM MEMBER CNTRIBUTIONS:

Team Member	Contribution
1. ANANYA DAS	C++ class design and core implementation
2. SANANDA CHOWDHURY MEDHA	Event registration and venue allocation module
3. K.KURMANADH	File handling, serialization and data persistence
4. B.LASYA SHARON	Testing, documentation and presentation

ADITYA UNIVERSITY

FUTURE ENHANCEMENTS

- Add a graphical user interface for easier interaction.
- Add login and role-based access for students and coordinators.
- Connect the system to a database such as MySQL.
- Add event search, filtering and sorting.
- Add email/SMS notifications for event registration.
- Generate downloadable attendance and event reports.
- Support multiple venues and richer scheduling conflict checks.

ADITYA UNIVERSITY

CONCLUSION

- The College Event Management System demonstrates how C++ and Object-Oriented Programming can be used to model and manage a real-world college event workflow. The system handles students, coordinators, venues, events, registration and attendance in one application.

- The project also provides practical experience with inheritance, abstraction, polymorphism, encapsulation, STL vectors, algorithms and file handling. It can be extended further with a GUI, database integration, authentication and advanced reporting.

ADITYA UNIVERSITY

THANK YOU

College Event Management System • C++ / OOP